ASSIGNMENT SET 5

1. Use linked list for polynomial representation and perform addition & subtraction operations on the polynomials.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int coeff, exp;
   struct node *next;
};

struct node *create(int c, int e) {
   struct node *n = malloc(sizeof(struct node));
   n->coeff = c;
   n->exp = e;
   n->next = NULL;
   return n;
}

void insert(struct node **head, int c, int e) {
   struct node *n = create(c, e);
   if (*head == NULL) {
      *head = n;
      return;
   }
   struct node *t = *head;
   while (t->next != NULL)
      t = t->next;
   t->next = n;
}

struct node *readPoly() {
   struct node *head = NULL;
   int n, c, e, i;
   printf("Enter number of terms: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
```

```
        printf("Enter coefficient and exponent: ");
        scanf("%d%d", &c, &e);
        insert(&head, c, e);
    }
    return head;
}

struct node *calculate(struct node *a, struct node *b, int sign) {
    struct node *r = NULL;
    while (a && b) {
        if (a->exp == b->exp) {
            insert(&r, a->coeff + sign * b->coeff, a->exp);
            a = a->next;
            b = b->next;
        } else if (a->exp > b->exp) {
            insert(&r, a->coeff, a->exp);
            a = a->next;
        } else {
            insert(&r, sign * b->coeff, b->exp);
            b = b->next;
        }
    }
    while (a) {
        insert(&r, a->coeff, a->exp);
        a = a->next;
    }
    while (b) {
        insert(&r, sign * b->coeff, b->exp);
        b = b->next;
    }
    return r;
}

void display(struct node *p) {
    while (p) {
        printf("%dx^%d", p->coeff, p->exp);
        if (p->next) printf(" + ");
        p = p->next;
    }
    printf("\n");
}
```

```
int main() {
   struct node *a, *b, *r;

   printf("Polynomial 1:\n");
   a = readPoly();
   printf("Polynomial 2:\n");
   b = readPoly();

   r = calculate(a, b, 1);
   printf("Addition: ");
   display(r);

   r = calculate(a, b, -1);
   printf("Subtraction: ");
   display(r);

   return 0;
}
```

OUTPUT:

```
Polynomial 2:
Enter number of terms: 2
Enter coefficient and exponent: 7 1
Enter coefficient and exponent: 9 0
Addition: 9x^1 + 12x^0
Subtraction: -5x^1 + -6x^0
```

2. Find Previous node of circular LL

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

int main() {
```

```
    struct node *head = NULL, *last, *temp, *n;
    int count, i, key;

    printf("Enter number of nodes: ");
    scanf("%d", &count);

    for (i = 0; i < count; i++) {
        n = malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &n->data);

        if (head == NULL) {
            head = n;
            n->next = head;
        } else {
            last = head;
            while (last->next != head)
                last = last->next;
            last->next = n;
            n->next = head;
        }
    }

    printf("Enter node value: ");
    scanf("%d", &key);

    temp = head;
    do {
        if (temp->next->data == key) {
            printf("Previous node = %d\n", temp->data);
            return 0;
        }
        temp = temp->next;
    } while (temp != head);

    printf("Node not found\n");
    return 0;
}
```

OUTPUT:

```
PS C:\Users\B.Cristina Jane
Enter number of nodes: 2
Enter data: 12
Enter data: 13
Enter node value: 13
Previous node = 12
```

3. Insert at a given position of a circular LL

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

int main() {
   struct node *head = NULL, *last, *temp, *n;
   int count, i, pos, value;

   printf("Enter number of nodes: ");
   scanf("%d", &count);

   for (i = 0; i < count; i++) {
      n = malloc(sizeof(struct node));
      scanf("%d", &n->data);

      if (head == NULL) {
         head = n;
         n->next = head;
      } else {
         last = head;
         while (last->next != head)
            last = last->next;
         last->next = n;
         n->next = head;
      }
   }

   printf("Enter position and value: ");
   scanf("%d%d", &pos, &value);
```

```
    if (pos < 1 || pos > count + 1) {
        printf("Invalid position");
        return 0;
    }

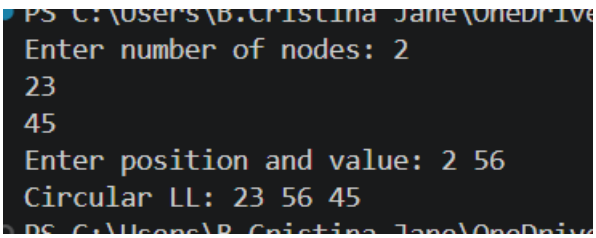
    n = malloc(sizeof(struct node));
    n->data = value;

    if (pos == 1) {
        last = head;
        while (last->next != head)
            last = last->next;
        n->next = head;
        last->next = n;
        head = n;
    } else {
        temp = head;
        for (i = 1; i < pos - 1; i++)
            temp = temp->next;
        n->next = temp->next;
        temp->next = n;
    }

    printf("Circular LL: ");
    temp = head;
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);

    return 0;
}
```

OUTPUT:



4. Find largest 2nd element in a queue

```
#include <stdio.h>

int main() {
   int q[100], n, i, largest, second;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   if (n < 2 || n > 100) {
      printf("Invalid number of elements");
      return 0;
   }

   printf("Enter queue elements: ");
   for (i = 0; i < n; i++)
      scanf("%d", &q[i]);

   largest = q[0];
   second = 0;

   for (i = 1; i < n; i++) {
      if (q[i] > largest) {
         second = largest;
         largest = q[i];
      } else if (q[i] < largest &&
               (second == 0 || q[i] > second)) {
```

```
            second = q[i];
        }
    }

    /* Use a separate flag to handle negative values. */
    {
        int found = 0;
        largest = q[0];

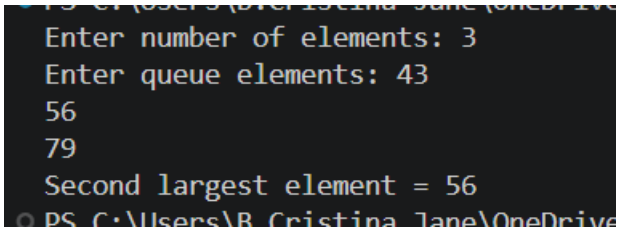
        for (i = 1; i < n; i++)
            if (q[i] > largest)
                largest = q[i];

        for (i = 0; i < n; i++)
            if (q[i] < largest &&
                (!found || q[i] > second)) {
                second = q[i];
                found = 1;
            }

        if (found)
            printf("Second largest element = %d", second);
        else
            printf("Second largest element does not exist");
    }

    return 0;
}
```

OUTPUT:



5. Insert 3rd element and check whether 1st and 3rd elements are same in a LL.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

int main() {
   struct node *head = NULL, *temp, *n;
   int count, i;

   printf("Enter number of nodes: ");
   scanf("%d", &count);

   for (i = 0; i < count; i++) {
      n = malloc(sizeof(struct node));
      printf("Enter data: ");
      scanf("%d", &n->data);
      n->next = NULL;

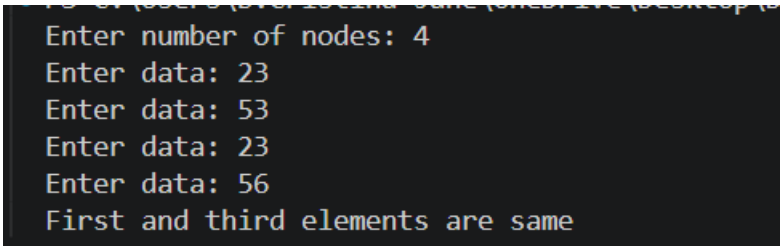
      if (head == NULL)
         head = n;
      else {
         temp = head;
         while (temp->next != NULL)
            temp = temp->next;
         temp->next = n;
      }
   }
```

```
    if (count < 3) {
        printf("At least 3 nodes are required");
        return 0;
    }

    if (head->data == head->next->next->data)
        printf("First and third elements are same");
    else
        printf("First and third elements are different");

    return 0;
}
```

OUTPUT:



6. Implement two stacks in one array. Write routines to traverse the array and display.

```
#include <stdio.h>
#define SIZE 10

int a[SIZE];
int top1 = -1, top2 = SIZE;

void push1(int x) {
    if (top1 + 1 == top2)
        printf("Stack Overflow\n");
    else
        a[++top1] = x;
}

void push2(int x) {
    if (top1 + 1 == top2)
        printf("Stack Overflow\n");
```

```
    else
        a[--top2] = x;
}

void pop1() {
    if (top1 == -1)
        printf("Stack 1 is empty\n");
    else
        printf("Popped element: %d\n", a[top1--]);
}

void pop2() {
    if (top2 == SIZE)
        printf("Stack 2 is empty\n");
    else
        printf("Popped element: %d\n", a[top2++]);
}

void display() {
    int i;

    printf("Stack 1: ");
    for (i = top1; i >= 0; i--)
        printf("%d ", a[i]);

    printf("\nStack 2: ");
    for (i = top2; i < SIZE; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int main() {
    int choice, stack, value;

    while (1) {
        printf("\n1. Push\n2. Pop\n3. Traverse and Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
```

```
            case 1:
                printf("Enter stack number (1 or 2): ");
                scanf("%d", &stack);
                printf("Enter value: ");
                scanf("%d", &value);

                if (stack == 1)
                    push1(value);
                else if (stack == 2)
                    push2(value);
                else
                    printf("Invalid stack number\n");
                break;

            case 2:
                printf("Enter stack number (1 or 2): ");
                scanf("%d", &stack);

                if (stack == 1)
                    pop1();
                else if (stack == 2)
                    pop2();
                else
                    printf("Invalid stack number\n");
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

OUTPUT:

```
1. Push
2. Pop
3. Traverse and Display
4. Exit
Enter choice: 1
Enter stack number (1 or 2): 1
Enter value: 34

1. Push
2. Pop
3. Traverse and Display
4. Exit
Enter choice: 1
Enter stack number (1 or 2): 2
Enter value: 56

1. Push
2. Pop
3. Traverse and Display
4. Exit
Enter choice: 2
Enter stack number (1 or 2): 1
Popped element: 34

1. Push
2. Pop
3. Traverse and Display
4. Exit
Enter choice: 1
Enter stack number (1 or 2): 1
Enter value: 34

1. Push
2. Pop
3. Traverse and Display
4. Exit
Enter choice: 3
Stack 1: 34
Stack 2: 56

1. Push
2. Pop
3. Traverse and Display
4. Exit
Enter choice: 4
```

7. Given a linked list containing n head nodes where every node in the LL contains two pointers - next pointer to next node in the LL- bottom pointer to a sub linked list where the current node is the head. Each of the sub-linked list nodes and head nodes are sorted in ascending order based on their data. We need to flatten the LL so that all the nodes appear in a single LL while maintaining sorted order.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
   struct node *bottom;
};

struct node *createNode(int data) {
   struct node *n = malloc(sizeof(struct node));
   n->data = data;
   n->next = NULL;
   n->bottom = NULL;
   return n;
}

struct node *merge(struct node *a, struct node *b) {
   struct node dummy;
   struct node *tail = &dummy;
   dummy.bottom = NULL;

   while (a != NULL && b != NULL) {
      if (a->data <= b->data) {
         tail->bottom = a;
         a = a->bottom;
      } else {
         tail->bottom = b;
         b = b->bottom;
      }

      tail = tail->bottom;
      tail->next = NULL;
   }
```

```
    if (a != NULL)
        tail->bottom = a;
    else
        tail->bottom = b;

    return dummy.bottom;
}

struct node *flatten(struct node *head) {
    if (head == NULL || head->next == NULL)
        return head;

    head->next = flatten(head->next);
    return merge(head, head->next);
}

int main() {
    struct node *head = NULL, *last = NULL;
    struct node *sub, *subLast, *temp;
    int n, m, i, j, x;

    printf("Enter number of head nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter number of nodes in sublist %d: ", i + 1);
        scanf("%d", &m);

        sub = NULL;
        subLast = NULL;

        printf("Enter sorted elements: ");

        for (j = 0; j < m; j++) {
            scanf("%d", &x);
            temp = createNode(x);

            if (sub == NULL)
                sub = temp;
            else
                subLast->bottom = temp;
```

```
            subLast = temp;
        }

        if (head == NULL)
            head = sub;
        else
            last->next = sub;

        last = sub;
    }

    head = flatten(head);

    printf("Output:\n");

    temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);

        if (temp->bottom != NULL)
            printf(" -> ");

        temp = temp->bottom;
    }

    printf("\n");
    return 0;
}
```

OUTPUT:

```
Enter number of head nodes: 2
Enter number of nodes in sublist 1: 2
Enter sorted elements: 56 78
Enter number of nodes in sublist 2: 2
Enter sorted elements: 90 100
Output:
56 -> 78 -> 90 -> 100
```

8. Given K sorted LL of different sizes, we need to merge them into a single list maintaining their sorted order.

```
#include <stdio.h>
```

```
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

struct node *create(int x) {
   struct node *n = malloc(sizeof(struct node));
   if (n == NULL) {
      printf("Memory allocation failed\n");
      exit(1);
   }
   n->data = x;
   n->next = NULL;
   return n;
}

struct node *sortList(struct node *head) {
   struct node *i, *j;
   int temp;

   for (i = head; i != NULL; i = i->next) {
      for (j = i->next; j != NULL; j = j->next) {
         if (i->data > j->data) {
            temp = i->data;
            i->data = j->data;
            j->data = temp;
         }
      }
   }
   return head;
}

struct node *merge(struct node *a, struct node *b) {
   struct node dummy;
   struct node *tail = &dummy;
   dummy.next = NULL;

   while (a != NULL && b != NULL) {
      if (a->data <= b->data) {
```

```
            tail->next = a;
            a = a->next;
        } else {
            tail->next = b;
            b = b->next;
        }
        tail = tail->next;
    }

    tail->next = (a != NULL) ? a : b;
    return dummy.next;
}

int main() {
    struct node **lists, *head = NULL, *last, *n;
    int k, size, i, j, x;

    printf("Enter number of lists: ");
    scanf("%d", &k);

    if (k <= 0) {
        printf("Invalid number of lists\n");
        return 0;
    }

    lists = malloc(k * sizeof(struct node *));
    if (lists == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (i = 0; i < k; i++) {
        lists[i] = NULL;
        last = NULL;

        printf("Enter size of list %d: ", i + 1);
        scanf("%d", &size);

        if (size < 0) {
            printf("Invalid list size\n");
            free(lists);
```

```
            return 0;
        }

        printf("Enter elements: ");

        for (j = 0; j < size; j++) {
            scanf("%d", &x);
            n = create(x);

            if (lists[i] == NULL)
                lists[i] = n;
            else
                last->next = n;

            last = n;
        }

        lists[i] = sortList(lists[i]);
    }

    for (i = 0; i < k; i++)
        head = merge(head, lists[i]);

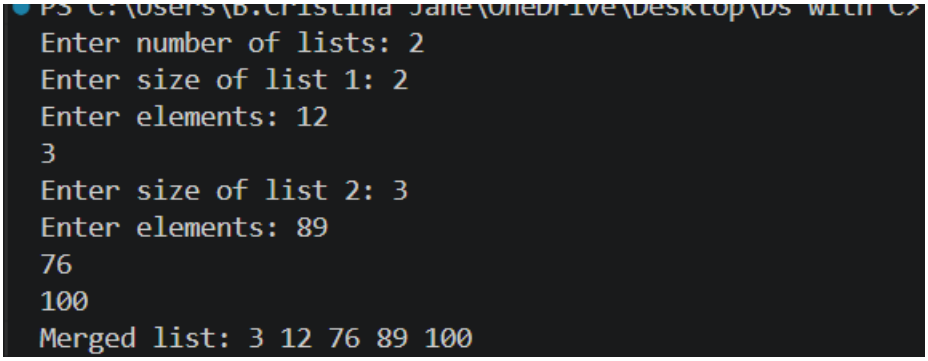
    printf("Merged list: ");

    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }

    printf("\n");
    free(lists);

    return 0;
}
```

OUTPUT:



9. Swap Kth node from beginning with Kth node from end in a LL.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

int main() {
   struct node *head = NULL, *last = NULL;
   struct node *temp, *x, *y, *px, *py, *t;
   int n, i, k, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
      scanf("%d", &value);

      temp = malloc(sizeof(struct node));
      temp->data = value;
      temp->next = NULL;

      if (head == NULL)
         head = temp;
      else
         last->next = temp;

      last = temp;
   }
```

```
printf("Enter k: ");
scanf("%d", &k);

if (k < 1 || k > n) {
   printf("Invalid k");
   return 0;
}

px = py = NULL;
x = y = head;

for (i = 1; i < k; i++) {
   px = x;
   x = x->next;
}

for (i = 1; i < n - k + 1; i++) {
   py = y;
   y = y->next;
}

if (x == y) {
   printf("Both positions are the same");
} else {
   if (px != NULL)
      px->next = y;
   else
      head = y;

   if (py != NULL)
      py->next = x;
   else
      head = x;

   t = x->next;
   x->next = y->next;
   y->next = t;
}

printf("List after swapping: ");
```

```
    temp = head;

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    return 0;
}
```

OUTPUT:

```
Enter number of nodes: 5
12 34 65 89 90
Enter k: 2
List after swapping: 12 89 65 34 90
```

10. Multiply two numbers represented by two LL.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n) {
    struct node *head = NULL, *last = NULL, *temp;
    int i, digit;

    for (i = 0; i < n; i++) {
        printf("Enter digit: ");
        scanf("%d", &digit);

        temp = malloc(sizeof(struct node));
        temp->data = digit;
        temp->next = NULL;

        if (head == NULL)
            head = temp;
```

```
        else
            last->next = temp;

        last = temp;
    }

    return head;
}

long long convert(struct node *head) {
    long long num = 0;

    while (head != NULL) {
        num = num * 10 + head->data;
        head = head->next;
    }

    return num;
}

int main() {
    struct node *a, *b;
    long long n1, n2, product;
    int n, i, digit;
    struct node *result = NULL, *temp;

    printf("Enter number of digits in first number: ");
    scanf("%d", &n);
    a = createList(n);

    printf("Enter number of digits in second number: ");
    scanf("%d", &n);
    b = createList(n);

    n1 = convert(a);
    n2 = convert(b);

    product = n1 * n2;

    printf("Product = %lld\n", product);
```

```
    if (product == 0) {
        printf("Result linked list: 0");
        return 0;
    }

    /* Insert each digit at the beginning. */
    while (product > 0) {
        digit = product % 10;

        temp = malloc(sizeof(struct node));
        temp->data = digit;
        temp->next = result;
        result = temp;

        product /= 10;
    }

    printf("Result linked list: ");

    while (result != NULL) {
        printf("%d", result->data);
        if (result->next != NULL)
            printf(" -> ");
        result = result->next;
    }

    return 0;
}
```

OUTPUT:

